Appendix: Artifact Description/Artifact Evaluation

Artifact Description (AD)

1 Overview of Contributions and Artifacts

1.1 Paper's Main Contributions

In this paper, we propose xGR, a serving system that meets the low-latency constraints under high-concurrency recommendation scenarios. The main contributions are as follows:

- C_1 We optimize the overall process spanning prefill, decode, and beam search phases. At the operator level, xAttention accelerates inference phases through KV cache management and staged attention computation. At the algorithm level, xBeam reduces search overhead with valid path constraint and early search termination.
- C_2 We deconstruct the serving pipeline to facilitate system-level optimizations. xSchedule employs a three-tier hierarchy to enable pipeline overlap and task parallelism.
- C_3 We develop a GR serving system xGR that enables fast recommendations under high-concurrency request scenarios. The experimental results show that xGR achieves higher throughput compared to the state-of-the-art baseline under strict P99 latency contraints.

1.2 Computational Artifacts

A_1 https://doi.org/YY.YYYY/zenodo.0XXXXX

Artifact ID	Contributions Supported	Related Paper Elements
A_1	C_1	Figure 16
A_1	C_2	Figure 15
A_1	C_3	Figures 13, 18

2 Artifact Identification

We integrate the computational artifact into a single DOI link A_1, where the corresponding contributions are shown in the table above. Artifact A_1 contains not only the source code of the xGR system but also scripts to reproduce the results. Note that the OneRec model and JD dataset are not readily available for open source. Instead, we will provide accounts for remote access during the AE stage.

2.1 Computational Artifact A_1

Relation To Contributions

- C_1 We jointly verify the effectiveness of operator-level and algorithm-level optimizations. We analyze fine-grained kernel efficiency of xAttention compared to PagedAttention on Ascend NPU (Figure 17). We set the beam width to {128, 256, 512}, batch size to {1, 4}, and input length to {1024, 2048}. We focus on three key metrics including kernel latency, computational throughput and memory access overhead.
- C_2 We jointly verify the overall effectiveness including system-level optimizations. We analyze the memory usage of xGR compared to xLLM using the Qwen3-4B on Ascend NPU, where the RPS is set to 4. For Figure 15(a), the beam width increases from 128 to 512 with a fixed input length of 1k tokens. For Figure 15(b), the input length increases from 500 to 3,000 with a fixed beam width of 256.
- C_3 We verify the effectiveness of the GR serving system xGR. First, we conduct the end-to-end experiments on the Ascend NPU cluster with Amazon Review dataset (Figure 13). The Qwen3 model ranges from 0.6B to 4B parameters. The beam width is set to {128, 256, 512}. Second, we extend the evaluation to NVIDIA GPU cluster to demonstrate the portability of our system with the same configurations (Figure 18).

Expected Results

- C_1 Regarding kernel latency and computational throughput, xAttention demonstrates superior performance compared to PagedAttention, particularly at large beam width. Regarding the memory access overhead, the pipeline spends most of PageAttention's time in a busy state. In contrast, xAttention maintains a significantly lower busy rate.
- C_2 Regarding the memory efficiency, xGR maintains a stable memory footprint when beam width increases, whereas xLLM exhibits super-linear growth. With the beam width fixed, xGR maintains a remarkably low footprint even with high input length, significantly outperforming xLLM.
- C_3 Regarding end-to-end performance, xGR consistently outperforms xLLM and vLLM across all cases on both Ascend NPU cluster and NVIDIA GPU cluster. Specifically, xGR sustains a significantly higher throughput under strict latency constraints (P99≤200ms). As the RPS increases, the baselines experience a sharp rise in latency and quickly reach the constraint. In contrast, xGR maintains a relatively smooth latency increase even at high loads.

Expected Reproduction Time (in Minutes)

Taking Ascend NPU cluster as an example, the expected reproduction time of artifact A_1 is about xx minutes. This includes xx minutes for C_1, xx minutes for C_2, and xx minutes for C_3.

Artifact Setup (incl. Inputs)

Hardware. We evaluate artifact A_1 on an Ascend cluster and a GPU cluster. The Ascend cluster comprises 4 nodes, each with 16 Ascend NPUs (64GB) interconnected via HCCS. The GPU cluster comprises 8 NVIDIA H800 GPUs (80GB) interconnected via NVLink.

Software. We conduct experiments in the software environment configured with vLLM vxx and xLLM vxx. We use the vllm-ascend porting for the Ascend cluster.

Datasets / Inputs. We use the publicly available dataset Amazon Review as input, and the models are multiple versions of the Qwen series, including Qwen3-0.6B, Qwen3-1.7B, and Qwen3-4B.

Installation and Deployment. To be supplemented ...

Artifact Execution

There are three tasks included in the Artifact Execution: conducting experiments to evaluation the collaborative performance of attention computation and beam search ((T_1), conducting experiments to evaluate the efficiency of scheduling optimization (T_2), conducting experiments to evaluate the end-to-end performance under strict P99 latency contraints (T_3). Each task involves actual program execution and result analysis. The execution scripts automatically call the plotting code in the /plot directory and saves the corresponding figures in the current directory.

In Task T_1, to be supplemented ...

In Task T_2, to be supplemented ...

In Task T_3, to be supplemented ...

In all, the execution flow of the artifact is: $T_1 \rightarrow T_2 \rightarrow T_3$.

Artifact Analysis (incl. Outputs)

2.2 Computational Artifact A_2

T_1 For the kernel efficiency, xAttention demonstrates superior performance regarding latency, throughput, and memory access overhead. The results indicates that xAttention effectively saturates the compute units, whereas PagedAttention suffers from idle stalls. In addition, xAttention successfully transforms the workload from memory-bound to compute-bound and maximizes hardware utilization.

T_2 For the memory efficiency, xGR maintains a stable memory footprint, significantly lower than that of xLLM. The excessive memory consumption in xLLM stems from the additional block copying inherent in PagedAttention. xGR addresses this through separated KV cache management, maintaining a single physical copy of the shared prefix and just enough space to store the decoded tokens.

T_3 For the end-to-end performance, xGR consistently outperforms xLLM and vLLM across all cases. The results demonstrate the efficacy of staged attention computation and KV cache separation. In addition, xGR leverages the high accelerator parallelism for both beam sorting and filtering, coupled with shared prefix management. By avoiding the redundant loading of shared prompt contexts, xGR supports high concurrency robustly, whereas the baselines frequently trigger preemption under similar conditions.